

The Relational Model

1. Core Building Blocks

- **Domain** — a set of **atomic** (indivisible) values, along with a name, data type/format, and optional constraints. Every attribute's values are drawn from some domain.
Domain "Emp-Age" = integers between 18 and 65 Domain "GPA" = floats between 0 and 4
- **Relation Schema** — $R(A_1, A_2, \dots, A_n)$: a relation **name** R plus a list of **attributes**, each of which is a name for a role played by some domain. The **degree** of a relation is the number of attributes in its schema.
- **Relation (state)** — an actual **set of n-tuples**, $r(R) = \{t_1, t_2, \dots, t_m\}$, conforming to schema R . Each tuple $t = \langle v_1, v_2, \dots, v_n \rangle$ is an ordered list of values, one per attribute, with $v_i \in \text{dom}(A_i)$.

Relation Schema: Student(Name, SSN, Home-phone)
Relation State (an actual snapshot at some point in time):

Name	SSN	Home-phone
Bayer	305-61-2189	373-1616
Chung	581-62-4455	375-4409

Schema vs. state: the schema changes **rarely** (only when the design is altered); the state (the actual data) changes **frequently** (every insert/update/delete). This is the exact same schema-vs-instance distinction covered in DBMS Fundamentals, applied specifically to one relation.

A relation is mathematically a subset of the Cartesian product of its attributes' domains:

$$r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$$

This immediately gives an upper bound on how large a relation *could* be: $|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times \dots \times |\text{dom}(A_n)|$ — the maximum possible number of distinct tuples.

2. Characteristics of Relations

1. **Ordering of tuples doesn't matter** — a relation is a mathematical **set**, so $\{t_1, t_2\}$ and $\{t_2, t_1\}$ represent the exact same relation.
2. **Ordering of values within a tuple, at the abstract level, doesn't matter either** — as long as attribute names are matched to values correctly (in practice, implementations do use positional ordering for efficiency, but conceptually, $t[\text{Name}]$ matters, not "the 2nd value").

3. **All attribute values are atomic** — no composite or multivalued attributes are allowed directly in a relation. **This assumption is exactly what "First Normal Form (1NF)" formalizes** (developed further in the Normalization topic) — it's baked into the relational model's very definition, not an optional extra rule.
4. **NULL values** — represent one of three distinct situations: value **unknown** (exists but not currently known), value **not applicable** (doesn't apply to this tuple at all), or value **not available/withheld**. NULLs deliberately don't distinguish *which* of these applies — a well-known practical wrinkle of the relational model.
5. **Interpretation of a relation** — a relation schema represents a general **assertion/predicate** about the real world (e.g., `Student(ID, Name)` asserts "every student has these attributes"); each individual **tuple represents a fact** — one specific instance that satisfies that assertion.

Why does 1NF-style atomicity matter enough to be baked in? If attributes could be composite/multivalued directly, operations like joins and the mathematical set-based foundation of relational algebra (see that topic) would become far messier to define cleanly — atomicity is what keeps the whole mathematical model, and by extension SQL's query semantics, tractable.

3. Key Constraints

Superkey, Key, Candidate Key, Primary Key

Superkey $\supseteq$ Key (a Key is always a Superkey, but not vice versa)

- **Superkey** — a set of attributes such that **no two distinct tuples** can have the same combination of values for those attributes. Every relation has at least one trivial superkey: the set of *all* its attributes.
- **Key** — a **minimal** superkey — removing *any* attribute from it destroys the uniqueness guarantee. A key is, by definition, a superkey with no redundant attributes.
- **Candidate key** — a relation may have **multiple** possible keys; each one is called a candidate key.
- **Primary key** — **one** candidate key, chosen (by the designer) as *the* main identifier for the relation. Primary key attributes **cannot be NULL** (this is the **Entity Integrity** constraint, below).

Worked example: `Employee(EmpID, Name, SSN)`

- `{EmpID}` → a key (unique, minimal).
- `{SSN}` → also a key (also unique, minimal) — so `{EmpID}` and `{SSN}` are both **candidate keys**.
- `{EmpID, Name}` → a **superkey** (still unique because `EmpID` alone guarantees it) but **not a key** — `Name` is redundant, since removing it doesn't break uniqueness.

Foreign Key

- A set of attributes **FK** in a **referencing relation** **R1** whose values must either (a) match the **primary key** value of some tuple in the **referenced relation** **R2**, or (b) be entirely **NULL**.
- This is exactly how relationships from the ER model get physically represented once mapped to the relational model (see §5).

4. Relational Integrity Constraints

Constraint	Rule
Domain constraint	Every attribute's value must be atomic and belong to that attribute's declared domain
Key constraint	No two tuples in a relation can have identical values for a (candidate) key
Entity integrity	No primary key attribute may be NULL, in any tuple, in any relation
Referential integrity	Every foreign key value must either match an existing primary key value in the referenced relation, or be entirely NULL

Why is entity integrity specifically about primary keys, not all candidate keys? The primary key is the relation's chosen *identity* — the value used to distinguish and reference a specific tuple (including by other relations' foreign keys). A NULL primary key would mean "this tuple has no identity," which breaks the ability to reliably reference or distinguish it at all — a severity that doesn't automatically extend to *other* (non-chosen) candidate keys, though good practice often still avoids NULLs there too.

Worked example — referential integrity: `Employee(..., Dno)` where `Dno` is a foreign key referencing `Department(Dnumber, ...)`. Every employee's `Dno` value must correspond to an actual, existing department's `Dnumber` — an employee can't claim to belong to a department that doesn't exist (unless `Dno` is NULL, meaning "not yet assigned to any department").

5. Handling Constraint Violations on Update Operations

Insert

Can violate **any** of domain, key, entity integrity, or referential integrity constraints (e.g., inserting a tuple with a foreign key that doesn't exist yet in the referenced relation).

Delete

Can only violate **referential integrity** (deleting a tuple that other tuples' foreign keys still point to).
Four options when this happens:

Option	Behavior
Reject	Refuse the delete outright
Cascade	Automatically delete all dependent (referencing) tuples too
Set NULL	Set the referencing foreign key values to NULL
Set default	Set the referencing foreign key values to some predefined default value

Worked example: deleting a **Department** that still has **Employee** tuples referencing it via **Dno** :

- **Reject** — refuse to delete the department until all its employees are reassigned/removed.
- **Cascade** — deleting the department also deletes all its employees (rarely desirable for this specific case!).
- **Set NULL** — employees' **Dno** becomes NULL ("currently unassigned").
- **Set default** — employees' **Dno** is reassigned to some designated default department.

Update

Can violate different constraints depending on **which** attribute is updated:

- Updating a **primary key** — can violate entity integrity (if set to NULL) or the key constraint (if set to a duplicate value).
- Updating a **foreign key** — same four options as delete (reject/cascade/set null/set default) apply.
- Updating an **ordinary attribute** — can only violate the domain constraint (if given an out-of-domain value).

Interview soundbite: "Insert is the most dangerous operation constraint-wise — it can violate every single type of constraint at once. Delete is comparatively narrow — it can only ever threaten referential integrity, because removing a tuple can't itself create a duplicate or an out-of-domain value; it can only orphan someone else's foreign key."

6. Beyond the Basic Constraints — Business Rules

- **Semantic integrity constraints** — rules that go beyond domain/key/referential integrity, reflecting real-world business logic (e.g., "an employee's salary cannot exceed their supervisor's salary"). These are **not** automatically enforced by the relational model itself — they typically require application-level enforcement via application programs, constraint/trigger languages, or database assertions.

- **State constraints** — define what conditions a *valid state* of the database must satisfy (e.g., "a department must have a unique department number" — actually just a restatement of the key constraint, but state constraints generalize this idea to arbitrary business rules).
 - **Transition constraints** — define rules about how the database is allowed to change from one state to another (e.g., "an employee's salary can only increase, never decrease") — usually enforced by application programs, since the relational model has no native way to compare "before" and "after" states directly.
-

Interview Questions With Answers

Q1. What's the difference between a relation schema and a relation state, and why does the distinction matter?

Answer: A relation schema is the structural definition — the relation's name and its list of attributes (e.g., `Student(Name, SSN, Phone)`) — and it changes rarely. A relation state is the actual set of tuples currently satisfying that schema at a given moment, and it changes constantly with every insert/update/delete. The distinction matters because it separates *what the data means and looks like structurally* from *what data currently happens to exist* — exactly mirroring the schema-vs-instance distinction that runs throughout DBMS design (the three-schema architecture, ER model type-vs-set, etc.).

Q2. Why is "all attribute values must be atomic" considered a foundational rule of the relational model rather than just a style preference?

Answer: The relational model is mathematically defined as a set of tuples drawn from the Cartesian product of attribute domains, where each domain is explicitly a set of *atomic* values. If attributes could hold composite or multivalued data directly, this clean set-theoretic foundation — and by extension, the well-defined semantics of relational algebra operations (joins, unions, etc., covered in that topic) — would become far messier to define and reason about. This atomicity requirement is formalized separately as First Normal Form (1NF) in the Normalization topic, but it's actually baked into the relational model's definition from the start, not an optional add-on.

Q3. Explain the relationship between superkey, candidate key, and primary key using a concrete example.

Answer: A superkey is any set of attributes guaranteeing tuple uniqueness (possibly with redundant attributes included) — every relation trivially has at least one superkey (all its attributes together). A candidate key is a *minimal* superkey — remove any attribute and uniqueness breaks. A relation can have multiple candidate keys (e.g., in `Employee(EmpID, Name, SSN)`, both `{EmpID}` and `{SSN}` are candidate keys, since each alone guarantees uniqueness and neither can be shrunk further). The primary key is simply whichever *one* candidate key the designer chooses as the relation's main,

official identifier — the choice between `{EmpID}` and `{SSN}` as primary key is a design decision, not something derivable from the data alone.

Q4. Why must primary key attributes never be NULL, but this restriction doesn't automatically extend to every candidate key?

Answer: The primary key is the relation's designated means of uniquely identifying and referencing a specific tuple — including being the target that other relations' foreign keys point to. A NULL primary key value would mean that tuple has no reliable identity at all, breaking the ability to distinguish it from other tuples or reference it externally, which is precisely what entity integrity is designed to prevent. Other (non-chosen) candidate keys aren't formally required to be NOT NULL by the entity integrity rule specifically, since they aren't the relation's designated identity mechanism — though in good practice, a true candidate key (something genuinely capable of uniquely identifying a tuple) usually shouldn't be NULL either, this just isn't enforced as a formal named constraint the way primary key nullability is.

Q5. Why can an INSERT operation potentially violate every type of relational constraint, while DELETE can only violate referential integrity?

Answer: An insert introduces an entirely new tuple with new values, so it can independently violate the domain constraint (an out-of-domain value), the key constraint (a duplicate key value), entity integrity (a NULL primary key), or referential integrity (a foreign key pointing to a non-existent tuple) — any of these can go wrong with brand-new data. A delete only removes an existing tuple; it can't introduce a duplicate, an out-of-domain value, or a NULL primary key, since it's not creating new data at all — the only thing a delete can break is *other* tuples' foreign keys that were relying on the now-deleted tuple still existing, which is exactly what referential integrity governs.

Q6. Compare the four options for handling a referential integrity violation on delete: reject, cascade, set null, set default.

Answer: Reject simply refuses the delete outright until the referencing tuples are dealt with first — the safest but least automated option. Cascade automatically deletes all tuples that reference the deleted tuple, propagating the deletion — powerful but risky if applied where it isn't semantically appropriate (e.g., deleting a department shouldn't necessarily delete all its employees). Set NULL updates referencing foreign keys to NULL, effectively saying "these tuples now reference nothing" — appropriate when NULL is a semantically valid state for that foreign key (e.g., "employee currently has no assigned department"). Set default reassigns referencing foreign keys to some predefined default value instead of NULL — useful when there's a sensible fallback (e.g., reassigning orphaned employees to a designated "unassigned" department) and NULL isn't desired or allowed.

Q7. Why are semantic (business rule) integrity constraints, like "an employee's salary cannot exceed their supervisor's salary," not automatically enforced by the relational model itself?

Answer: The relational model's built-in constraints (domain, key, entity integrity, referential integrity) are deliberately generic, structural rules that apply to *any* relational database regardless of what it actually represents — they check things like uniqueness and valid references, not domain-specific business logic. A rule like "salary ≤ supervisor's salary" requires comparing values across specific, related tuples according to meaning that's entirely particular to this one application's business rules — there's no generic, structural way for the relational model itself to know or enforce this, so it has to be enforced by mechanisms built on top of it: application code, database triggers/assertions, or constraint languages specifically written to encode that business rule.

Q8. Scenario: A relation `Order(OrderID, CustomerID, ProductID, Quantity)` has `CustomerID` as a foreign key referencing `Customer(CustomerID)`. A request comes in to delete a customer who has existing orders. Walk through what happens under each of the four referential integrity violation-handling policies, and recommend which is most appropriate for this specific business scenario.

Answer: Under **Reject**, the delete is simply refused — the customer can't be removed while they still have orders on file. Under **Cascade**, deleting the customer would also delete all of their orders — likely undesirable here, since order history often needs to be retained for accounting/auditing purposes even after a customer account is removed. Under **Set NULL**, the orders' `CustomerID` becomes NULL, preserving the order records but severing the link to any customer — problematic if `CustomerID` is expected to always identify who placed an order (and NULL might not even be allowed if `CustomerID` is itself required to be NOT NULL for business reasons). Under **Set default**, `CustomerID` could be reassigned to a designated placeholder value like a "deleted/anonymous customer" record — this preserves order history in a referentially consistent way while clearly indicating the original customer is gone. For this scenario, **Set default** (pointing to a placeholder "deleted customer" record) is typically the most appropriate — it retains historical order data intact (unlike Cascade), keeps referential integrity fully valid (unlike Set NULL, if NULL isn't semantically clean here), and doesn't block the legitimate business need to remove a customer's active account (unlike Reject).